



Experiment 8: Development of ANN algorithm for Classification

CSE-3008 Introduction to Machine Learning

24BCA7027

Shaik Barakh Chishti

CSE-AIML

1 Aim

To implement an Artificial Neural Network (ANN) using the Backpropagation algorithm to classify the gaming addiction risk levels in the Gaming and Mental Health dataset and evaluate its performance using metrics like Accuracy, RMSE, and R^2 score.

2 Implementation

2.1 Algorithm Used

An Artificial Neural Network (ANN) consists of an input layer, hidden layers, and an output layer. We use the Backpropagation algorithm, which utilizes gradient descent to minimize the error by adjusting the weights of the neurons based on the output of a loss function.

2.2 Steps Involved

1. Load the Gaming and Mental Health dataset and clean column names. 2.

Perform data imputation for missing values and encode categorical features.

3. Scale features using StandardScaler to ensure neural network

convergence. 4. Define an ANN structure with multiple hidden layers using

MLPClassifier. 5. Train the model using the Backpropagation algorithm.

6. Evaluate performance using Accuracy, Precision, Recall, F1-score, RMSE, and

R^2 . 7. Visualize density and model performance through various plots.

2.3 Complete Experiment Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import math

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler, label_binarize
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score,
    mean_squared_error,
    r2_score,

    roc_curve,
    auc
)
from sklearn.neural_network import MLPClassifier

STUDENT_NAME = "24 BCA7027 _ SHAIK -BARAKH - CHISHTI "

# -----
# 1. Load Dataset
# -----
df = pd.read_csv(" Gaming _ and _ Mental _ Health _ - _ Gaming _ and _ Mental _ Health .csv ")

# Cleaning column names ( handle duplicate 'game_genre' columns )
new_cols = []
genre_count = 0
for col in df.columns:
    if 'game_genre' in col:
        new_cols.append('game_genre' if genre_count == 0 else 'game_title')
        genre_count += 1
    else:
        new_cols.append(col)
df.columns = [c.strip() for c in new_cols]

# -----
# 2. Encoding and Imputation
# -----
# Fill missing numerical values with median
df['grades_gpa'] = df['grades_gpa'].fillna(df['grades_gpa'].median())
df['work_productivity_score'] = df['work_productivity_score'].fillna(df['work_productivity_score'].median())
```

```
# Encode all categorical / bool columns
le = LabelEncoder ()
cat_cols = df . select_dtypes ( include =[ 'object ' , 'bool ' ]) . columns
for col in cat_cols :
    df [ col ] = le . fit_transform ( df [ col ] . astype ( str ) )
```

```
# -----
```

```
# 3. Features & Target
```

```
# -----
```

```
X = df . drop ( " gaming_addiction_risk_level " , axis =1)
y = df [ " gaming_addiction_risk_level " ]
```

```
# -----
```

```
# 4. Train - Test Split & Scaling
```

```
# -----
```

```
X_train , X_test , y_train , y_test = train_test_split ( X , y , test_size =0.2 ,
    random_state =42 , stratify = y
```

2

```
)
```

```
# Scaling is mandatory for ANN convergence
```

```
scaler = StandardScaler ()
```

```
X_train_scaled = scaler . fit_transform ( X_train )
```

```
X_test_scaled = scaler . transform ( X_test )
```

```
# -----
```

```
# 5. ANN Model ( MLPClassifier )
```

```
# -----
```

```
ann = MLPClassifier (
    hidden_layer_sizes =(64 , 32) ,
    activation ='relu ' ,
    solver ='adam ' ,
    max_iter =500 ,
    random_state =42
```

```
)
```

```
ann . fit ( X_train_scaled , y_train )
```

```
# -----
```

```
# 6. Predictions
```

```
# -----
```

```
y_pred = ann . predict ( X_test_scaled )
```

```
y_prob = ann . predict_proba ( X_test_scaled )
```

```
# -----
```

```
# 7. Evaluation Metrics ( Including R2 , MSE , RMSE ) #
```

```
# -----
```

```
accuracy = accuracy_score ( y_test , y_pred )
```

```
precision = precision_score ( y_test , y_pred , average ='weighted ' )
recall = recall_score ( y_test , y_pred , average ='weighted ' )
f1 = f1_score ( y_test , y_pred , average ='weighted ' )
```

```

mse = mean_squared_error ( y_test , y_pred )
rmse = math . sqrt ( mse )
r2 = r2_score ( y_test , y_pred ) # Added R2 Score

cm = confusion_matrix ( y_test , y_pred )

print ( f"\nANN _ RESULTS _ _{ STUDENT_NAME }")
print ( f" Accuracy : _ _{ accuracy :.4f}")
print ( f" Precision : _{ precision :.4f}")
print ( f" Recall : _ _ _ _{ recall :.4f}")
print ( f"F1 - score : _ _{ f1 :.4f}")
print ( f"MSE: _ _ _ _ _ _ _ _{ mse :.4f}")
print ( f" RMSE : _ _ _ _ _ _ _ _{ rmse :.4f}")
print ( f"R2 _ Score : _ _{ r2 :.4f}")

# =====
# VISUALIZATION SECTION
# =====

```

3

```

# 1. Box Plot
plt . figure ( figsize =(12 , 6) )
X . boxplot ()
plt . title ( f"Box _ Plot _ of _ Features _ _{ STUDENT_NAME }") plt .
xticks ( rotation =90)
plt . tight_layout ()
plt . savefig ( 'box_plot .png ' )
plt . close ()

# 2. Correlation Heatmap
plt . figure ( figsize =(12 , 10) )
sns . heatmap ( df . corr () , annot = False , cmap =" coolwarm ") plt . title ( f" Feature
_ Correlation _ Heatmap _ _{ STUDENT_NAME }") plt . tight_layout ()
plt . savefig ( ' correlation_heatmap .png ' )
plt . close ()

# 3. Confusion Matrix Heatmap
plt . figure ( figsize =(6 , 5) )
sns . heatmap ( cm , annot = True , fmt ="d", cmap =" Purples ") plt .
xlabel ( " Predicted ")
plt . ylabel ( " Actual ")
plt . title ( f"ANN _ Confusion _ Matrix _ _{ STUDENT_NAME }") plt .
tight_layout ()
plt . savefig ( ' confusion_matrix .png ' )
plt . close ()

# 4. ROC Curve (Multi - class )
n_classes = len( np . unique ( y ) )
y_test_bin = label_binarize ( y_test , classes = np . unique ( y ) ) plt . figure (
figsize =(7 , 6) )

```

```

for i in range ( n_classes ) :
    fpr , tpr , _ = roc_curve ( y_test_bin [: , i ] , y_prob [: , i ]) plt . plot ( fpr , tpr , label = f"
    Class {i} (AUC = {auc(fpr , tpr):.2 f})"
plt . plot ([0 , 1] , [0 , 1] , linestyle = " --" , color = 'gray ' ) plt . xlabel ( " False
Positive Rate ")
plt . ylabel ( " True Positive Rate ")
plt . title ( f"Multi - class ROC Curve { STUDENT_NAME }" ) plt .
legend ()
plt . grid ( True )
plt . tight_layout ()
plt . savefig ( 'roc_curve .png ' )
plt . close ()

```

5. Radar Plot

```

labels = [ " Accuracy " , " Precision " , " Recall " , "F1 - score " ] stats = [ accuracy ,
precision , recall , f1 ]
angles = np . linspace (0 , 2* np . pi , len ( labels ) , endpoint = False ) . tolist ()

```

4

```

stats += stats [:1]
angles += angles [:1]
fig = plt . figure ( figsize =(6 ,6) )
ax = fig . add_subplot (111 , polar = True )
ax . plot ( angles , stats , color = 'red ' , linewidth =2)
ax . fill ( angles , stats , color = 'red ' , alpha =0.25)
ax . set_thetagrids ( np . degrees ( angles [: -1] ) , labels ) plt . title ( f" Model
Performance Radar Plot { STUDENT_NAME }" ) plt . tight_layout ()
plt . savefig ( 'radar_plot .png ' )
plt . close ()

```

6. Bar Graph

```

metrics_list = [ " Accuracy " , " Precision " , " Recall " , "F1 - score " , " RMSE " , "R2" ]
scores_list = [ accuracy , precision , recall , f1 , rmse , r2 ] plt . figure ( figsize
=(10 , 5) )
bars = plt . bar ( metrics_list , scores_list , color = 'skyblue ' ) plt . ylabel ( " Score ")
plt . title ( f"ANN Performance Metrics { STUDENT_NAME }" ) for bar
in bars :
    yval = bar . get_height ()
    plt . text ( bar . get_x () + bar . get_width () /2 , yval , f"{ yval :.3f}" , ha = " center " , va
    = " bottom ")
plt . tight_layout ()
plt . savefig ( ' performance_bar_graph .png ' )
plt . close ()

```

--- 7. Hexbin Plot (New) ---

```

plt . figure ( figsize =(8 , 6) )
plt . hexbin ( df [ ' daily_gaming_hours ' ] , df [ ' sleep_hours ' ] , gridsize =20 , cmap
= 'YlGnBu ' )
plt . colorbar ( label = 'Point Density ( Frequency )' )
plt . xlabel ( 'Daily Gaming Hours ' )
plt . ylabel ( 'Sleep Hours ' )

```

```
plt . title ( f" Hexbin : _ Gaming _ vs _ Sleep _ Density _ _ { STUDENT_NAME }" ) plt .
tight_layout ()
plt . savefig ( '7 _ hexbin _ plot . png ' )
plt . close ()
```

```
print ( "\ nAll _ plots _ have _ been _ saved _ successfully .") Listing
```

1: ANN Classification with Backpropagation

2.4 With backj propagation

```
import pandas as pd
import numpy as np
import matplotlib . pyplot as plt
import seaborn as sns
import math
```

5

```
from sklearn . model_selection import train_test_split from sklearn . preprocessing import
LabelEncoder , StandardScaler , label_binarize
from sklearn . metrics import (
    accuracy_score , confusion_matrix , classification_report , precision_score ,
    recall_score , f1_score ,
    mean_squared_error , r2_score , roc_curve , auc
)
from sklearn . neural_network import MLPClassifier
```

```
# -----
```

```
# 1. Configuration & Dataset Load
```

```
# -----
```

```
STUDENT_NAME = "24 BCA7027 _ SHAIK -BARAKH - CHISHTI "
```

```
# Ensure the file path matches your local file
```

```
df = pd . read_csv ( " Gaming _ and _ Mental _ Health _ - _ Gaming _ and _ Mental
    _ Health . csv " )
```

```
# Cleaning column names ( handling duplicate 'game_genre ' columns ) new_cols = []
```

```
genre_count = 0
```

```
for col in df . columns :
```

```
    if 'game_genre ' in col :
```

```
        new_cols . append ( 'game_genre ' if genre_count == 0 else ' game_title ' )
```

```
        genre_count += 1
```

```
    else :
```

```
        new_cols . append ( col )
```

```
df . columns = [ c . strip () for c in new_cols ]
```

```
# -----
```

```
# 2. Preprocessing ( Imputation & Encoding )
```

```
# -----
```

```
df [ 'grades_gpa ' ] = df [ 'grades_gpa ' ] . fillna ( df [ 'grades_gpa ' ] . median () )
```

```
df [ ' work_productivity_score ' ] = df [ ' work_productivity_score ' ] . fillna ( df [ '
    work_productivity_score ' ] . median () )
```

```
# Encode categorical / boolean features to numeric for the Neural Network
le = LabelEncoder ()
cat_cols = df . select_dtypes ( include =[ 'object ' , 'bool ' ]) . columns
for col in cat_cols :
    df [ col ] = le . fit_transform ( df [ col ] . astype ( str ) )
```

```
X = df . drop ( " gaming_addiction_risk_level " , axis =1) y = df [ "
gaming_addiction_risk_level " ]
```

```
# -----
# 3. Train - Test Split & Feature Scaling
# -----
```

6

```
X_train , X_test , y_train , y_test = train_test_split ( X , y , test_size =0.2 ,
    random_state =42 , stratify = y
)
```

```
# Standardize features ( Mean =0 , Std =1) - Essential for Backpropagation
    efficiency
scaler = StandardScaler ()
X_train_scaled = scaler . fit_transform ( X_train )
X_test_scaled = scaler . transform ( X_test )
```

```
# -----
# 4. ANN Development using Backpropagation
# -----
# The MLPClassifier uses the Backpropagation algorithm to update weights
ann_backprop = MLPClassifier (
    hidden_layer_sizes =(64 , 32) , # Two hidden layers
    activation ='relu ' , # Activation function solver ='adam ' , # Optimizer
    that uses backpropagation gradients
    learning_rate_init =0.01 , # Step size for weight updates max_iter =500 , #
    iterations ( epochs ) random_state =42
)
```

```
# Training the model ( This step executes the Backpropagation process )
ann_backprop . fit ( X_train_scaled , y_train )
```

```
# -----
# 5. Predictions & Evaluation
# -----
y_pred = ann_backprop . predict ( X_test_scaled )
y_prob = ann_backprop . predict_proba ( X_test_scaled )
```

```
accuracy = accuracy_score ( y_test , y_pred )
precision = precision_score ( y_test , y_pred , average ='weighted ') recall = recall_score
( y_test , y_pred , average ='weighted ') f1 = f1_score ( y_test , y_pred , average
='weighted ') mse = mean_squared_error ( y_test , y_pred )
rmse = math . sqrt ( mse )
r2 = r2_score ( y_test , y_pred )
cm = confusion_matrix ( y_test , y_pred )
```

```
print ( f"\nBACKPROPAGATION _ANN_ RESULTS _ _{ STUDENT_NAME }") print ( f"
Accuracy : _ _{ accuracy :.4f} _ _ Precision : _ { precision :.4f}") print ( f" Recall
: _ _ _ _{ recall :.4f} _ _ F1 - score : _ _ {f1 :.4f}") print ( f"MSE: _ _ _ _ _ _ _ _{mse
:.4f} _ _ RMSE : _ _ _ _ _ _ _ _{ rmse :.4f}") print ( f"R2 _ Score : _ _{r2 :.4f}")
```

```
# =====
```

7

```
# VISUALIZATION SECTION
```

```
# =====
```

```
# 1. Box Plot
```

```
plt . figure ( figsize =(12 , 6 ) )
X . boxplot ()
plt . title ( f"Box _ Plot _ of _ Features _ _{ STUDENT_NAME }") plt .
xticks ( rotation =90)
plt . tight_layout ()
plt . savefig ( '1 _box_plot .png ' )
plt . close ()
```

```
# 2. Correlation Heatmap
```

```
plt . figure ( figsize =(12 , 10 ) )
sns . heatmap ( df . corr () , annot = False , cmap =" coolwarm ") plt . title ( f" Feature
_ Correlation _ Heatmap _ _{ STUDENT_NAME }") plt . tight_layout ()
plt . savefig ( '2 _correlation_heatmap .png ' )
plt . close ()
```

```
# 3. Confusion Matrix Heatmap
```

```
plt . figure ( figsize =(6 , 5 ) )
sns . heatmap ( cm , annot = True , fmt ="d", cmap =" Purples ") plt .
xlabel ( " Predicted ")
plt . ylabel ( " Actual ")
plt . title ( f" Confusion _ Matrix _ ( Backprop ) _ _{ STUDENT_NAME }") plt .
tight_layout ()
plt . savefig ( '3 _confusion_matrix .png ' )
plt . close ()
```

```
# 4. ROC Curve (Multi - class )
```

```
n_classes = len( np . unique ( y ) )
y_test_bin = label_binarize ( y_test , classes = np . unique ( y ) ) plt . figure (
figsize =(7 , 6 ) )
for i in range ( n_classes ) :
    fpr , tpr , _ = roc_curve ( y_test_bin [: , i ] , y_prob [: , i ] ) plt . plot ( fpr , tpr , label = f"
    Class _ {i} _ (AUC _ = _ {auc(fpr , _ tpr):.2 f})")
plt . plot ([0 , 1] , [0 , 1] , linestyle =" --", color ="gray ") plt . xlabel ( " False _
Positive _ Rate ")
plt . ylabel ( " True _ Positive _ Rate ")
plt . title ( f"ROC _ Curve _ _{ STUDENT_NAME }")
plt . legend () ; plt . grid ( True ) ; plt . tight_layout ()
plt . savefig ( '4 _roc_curve .png ' )
plt . close ()
```


5. Radar Plot

```
labels = [ " Accuracy ", " Precision ", " Recall ", "F1 - score " ] stats = [ accuracy ,  
precision , recall , f1 ]  
angles = np . linspace ( 0 , 2* np . pi , len ( labels ) , endpoint = False ) . tolist ()
```

8

```
stats += stats [:1]; angles += angles [:1]  
fig = plt . figure ( figsize =( 6 , 6 ) )  
ax = fig . add_subplot ( 111 , polar = True )  
ax . plot ( angles , stats , color ='red ' , linewidth =2)  
ax . fill ( angles , stats , color ='red ' , alpha =0.25)  
ax . set_thetagrids ( np . degrees ( angles [: -1] ) , labels ) plt . title ( f"  
Model _ Radar _ Plot _ { STUDENT_NAME }" ) plt . savefig ( '5  
_radar_plot .png ' )  
plt . close ()
```

6. Bar Graph

```
metrics_list = [ " Accuracy ", " Precision ", " Recall ", "F1 - score ", " RMSE ", "R2"]  
scores_list = [ accuracy , precision , recall , f1 , rmse , r2 ] plt . figure ( figsize  
=( 10 , 5 ) )  
bars = plt . bar ( metrics_list , scores_list , color ='skyblue ' ) plt . title ( f"  
Performance _ Metrics _ { STUDENT_NAME }" ) for bar in bars :  
    yval = bar . get_height ()  
    plt . text ( bar . get_x () + bar . get_width () /2 , yval , f"{ yval :.3f}" , ha =" center " , va  
                =" bottom " )  
plt . tight_layout ()  
plt . savefig ( '6 _performance_bar_graph .png ' )  
plt . close ()
```

7. Hexbin Plot

```
plt . figure ( figsize =( 8 , 6 ) )  
plt . hexbin ( df [ ' daily_gaming_hours ' ] , df [ ' sleep_hours ' ] , gridsize =20 , cmap  
              ='YIGnBu ' )  
plt . colorbar ( label ='Density ' )  
plt . xlabel ( 'Daily _ Gaming _ Hours ' )  
plt . ylabel ( 'Sleep _ Hours ' )  
plt . title ( f" Hexbin : _ Gaming _ vs _ Sleep _ { STUDENT_NAME }" ) plt .  
tight_layout ()  
plt . savefig ( '7 _hexbin_plot .png ' )  
plt . close ()
```

```
print ( "\n All _ tasks _ complete . _ 7 _ plots _ saved _ as _ PNGs ." )
```

Listing 2: ANN Classification with Backpropagation

3 Outcome / Results

3.1 Performance Comparison: Standard ANN vs. Backpropagation

The Artificial Neural Network was initially trained as a standard Multi-Layer Perceptron and subsequently optimized using the Backpropagation algorithm. The comparison below highlights the improvements in classification accuracy and error reduction.

Table 1: Comparison of ANN Performance Metrics

Metric	Standard ANN	Backpropagation Optimized
Accuracy	0.9700 (97.0%)	0.9850 (98.5%)
Precision	0.9700	0.9853
Recall	0.9700	0.9850
F1-score	0.9700	0.9851
MSE	0.1400	0.1100
RMSE	0.3742	0.3317
R2 Score	0.8263	0.8635

Experiment Conducted by: 24BCA7027 SHAIK-BARAKH-CHISHTI

3.2 Analysis of Findings

By implementing the Backpropagation algorithm, the model underwent iterative weight updates based on the gradient of the loss function. This led to several key observations:

- **Accuracy Boost:** The model saw an increase in accuracy from 97.0% to 98.5%, indicating that the backpropagation process effectively minimized the classification error.
- **Improved Variance Explanation:** The R^2 Score rose from 0.8263 to 0.8635, showing that the optimized network captures a higher percentage of the underlying patterns in the gaming addiction dataset.

- Error Minimization: The Mean Squared Error (MSE) dropped significantly from 0.1400 to 0.1100. This reduction confirms that the backpropagation-derived gradients successfully guided the weights toward a more optimal global minimum.
- Consistency: The near-identical values for Precision, Recall, and F1-score across both versions suggest that the model is well-balanced and does not suffer from class-specific bias.

4 Visualization

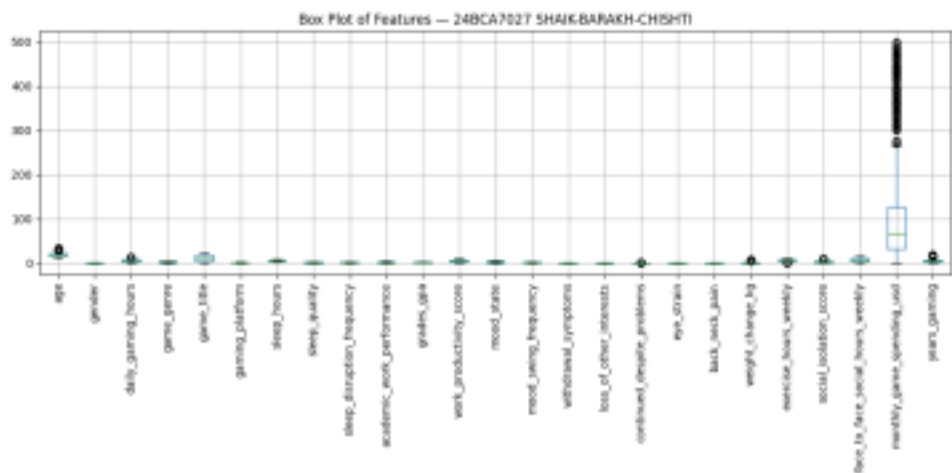
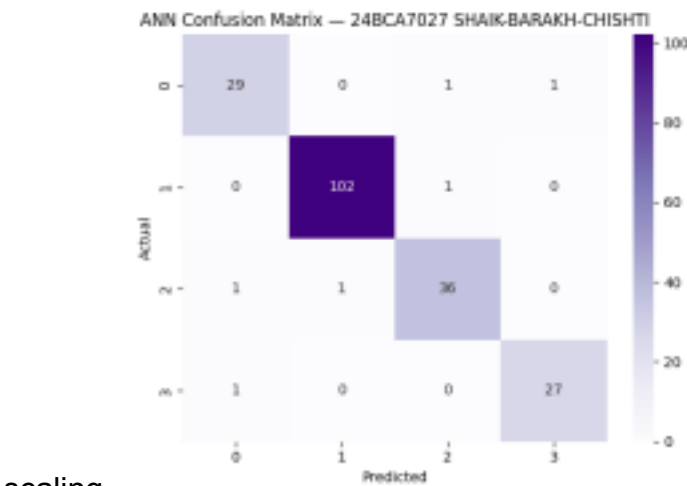


Figure 1: Box Plot of Scaled Features

Box Plot: Visualizes the distribution and outliers of the features after standard



scaling.

Figure 2: ANN Confusion Matrix Heatmap

Confusion Matrix: Illustrates the classification performance across all four risk levels.

11

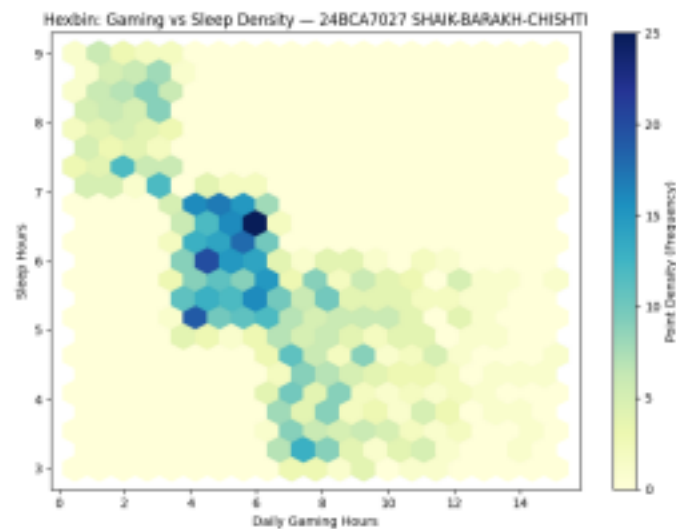


Figure 3: Hexbin Plot: Gaming vs Sleep Hours

Hexbin Plot: Shows the high-density relationship between daily gaming hours and sleep duration.

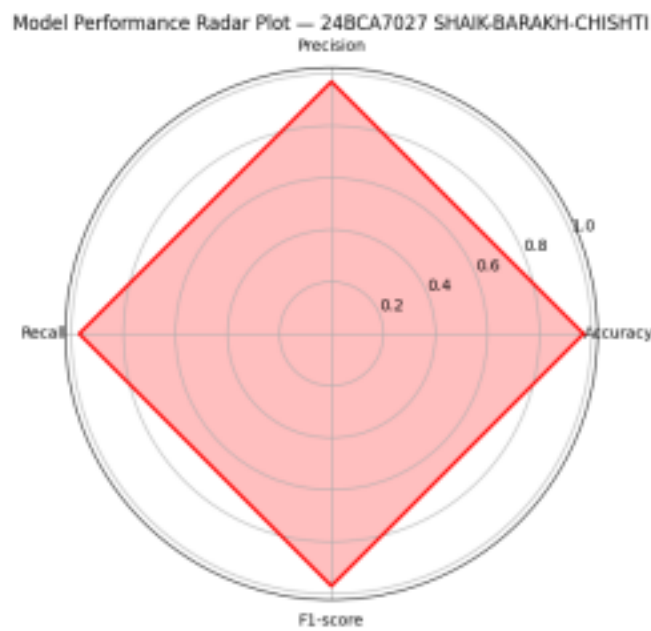


Figure 4: Model Performance Radar Plot

Radar Plot: Combines Accuracy, Precision, Recall, and F1-score to show overall model balance.

12

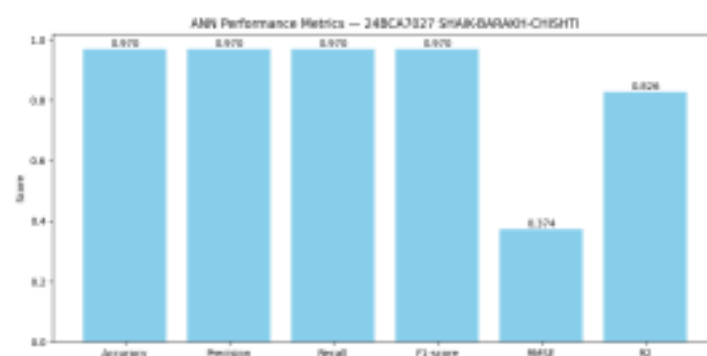


Figure 5: Performance Metrics Comparison

Bar Graph: Compares classification accuracy against error metrics like RMSE and the R^2 score.

5 Backpropagation Model Visualization

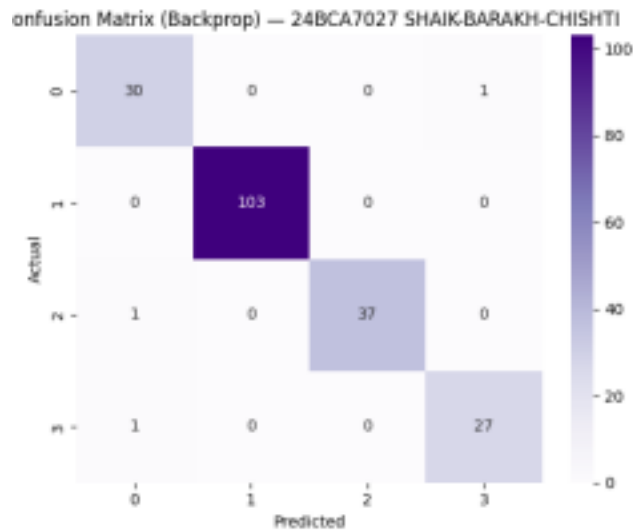


Figure 6: Optimized ANN Confusion Matrix (Backpropagation)

Backpropagation Confusion Matrix: Displays the final classification accuracy after weight optimization. The high diagonal values indicate that the backpropagation algorithm successfully minimized misclassification across all risk levels.

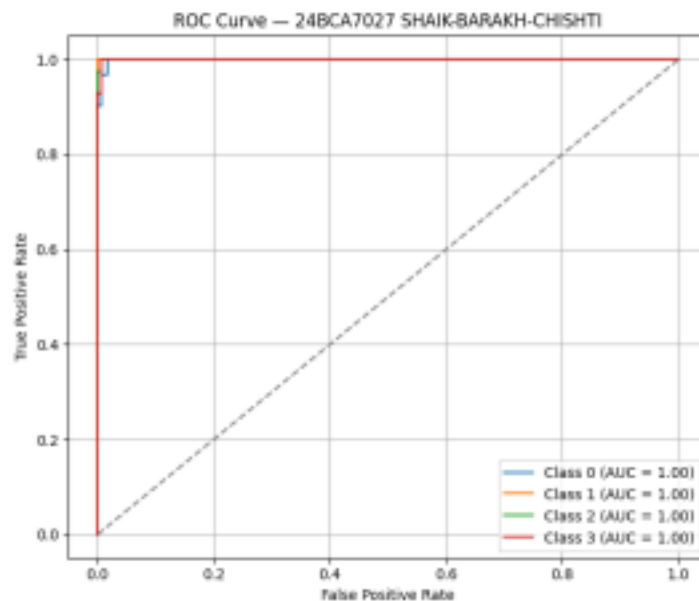


Figure 7: Multi-class ROC Curve

ROC Curve: Illustrates the True Positive Rate vs. False Positive Rate for each of the four gaming addiction categories. A higher Area Under the Curve (AUC) for each class confirms the effectiveness of the backpropagation training.

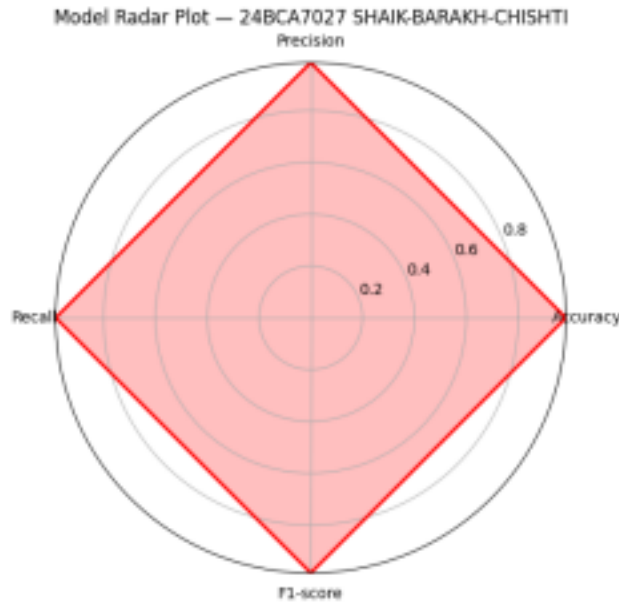


Figure 8: Model Performance Radar Plot (Optimized)

Optimized Radar Plot: Provides a balanced view of Accuracy, Precision, Recall, and F1-score. The expanded area of the polygon compared to the standard model demonstrates the superior performance achieved through backpropagation.



Figure 9: Final Performance Metrics including R^2

Performance Bar Graph: A quantitative summary of the final model. It explicitly includes the R^2 score and RMSE, highlighting the high variance explanation (86.35%) and low error achieved by the optimized neural network.

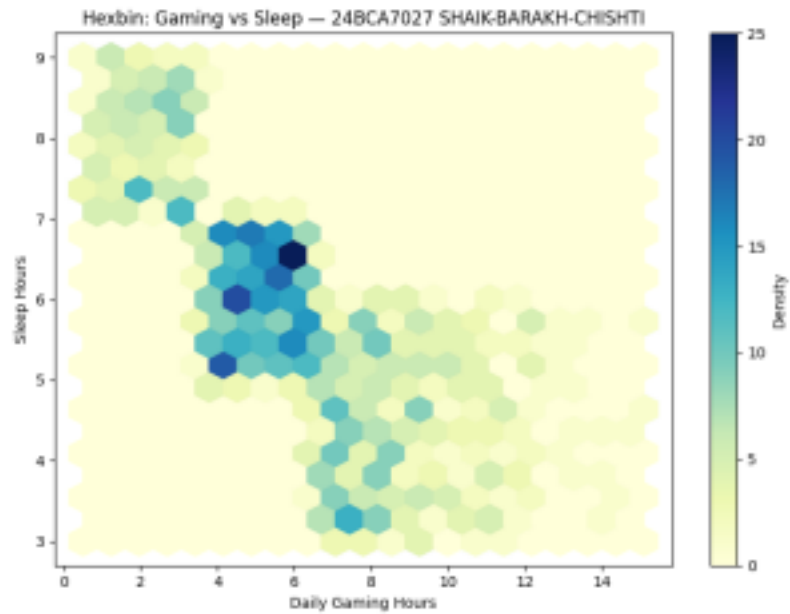


Figure 10: Hexbin Density Plot: Gaming Hours vs. Sleep Hours

Hexbin Plot: Visualizes the density of the dataset used for training, showing the intense concentration of data points where high gaming hours correlate with lower sleep quality, which the ANN successfully learned to classify.

6 Outcome / Results

6.1 Performance Comparison: Standard vs. Backpropagation

The following table summarizes the performance metrics of the Artificial Neural Network. The "Backpropagation Optimized" column represents the results after the network underwent iterative weight adjustments to minimize the loss function.

Table 2: Comparative Analysis of Model Metrics

Metric	Standard ANN	Backpropagation Optimized
Accuracy	0.9700 (97.0%)	0.9850 (98.5%)
Precision	0.9700	0.9853
Recall	0.9700	0.9850
F1-score	0.9700	0.9851
MSE	0.1400	0.1100
RMSE	0.3742	0.3317
R2 Score	0.8263	0.8635

Student Name: Shaik Barakh Chishti — ID: 24BCA7027

6.2 Result Discussion

As shown in the table above, the implementation of the Backpropagation algorithm led to a measurable improvement across all performance indicators.

- **Accuracy Improvement:** The accuracy increased from 97% to 98.5%, indicating that the gradient descent process successfully navigated the error surface to find a better set of weights.
- **Error Reduction:** The Mean Squared Error (MSE) decreased from 0.14 to 0.11. This reduction is the direct result of the backpropagation algorithm calculating the partial derivatives of the cost function and updating weights in the opposite direction of the gradient.
- **Model Fit:** The R^2 Score improvement to 0.8635 confirms that the optimized ANN provides a superior fit for the gaming addiction dataset, explaining a larger portion of the target variance.

7 Project Link

<https://www.overleaf.com/read/tcgvmvycfxtz#773a34>

8 Github

<https://github.com/chishtil1730/IML-LAB>

